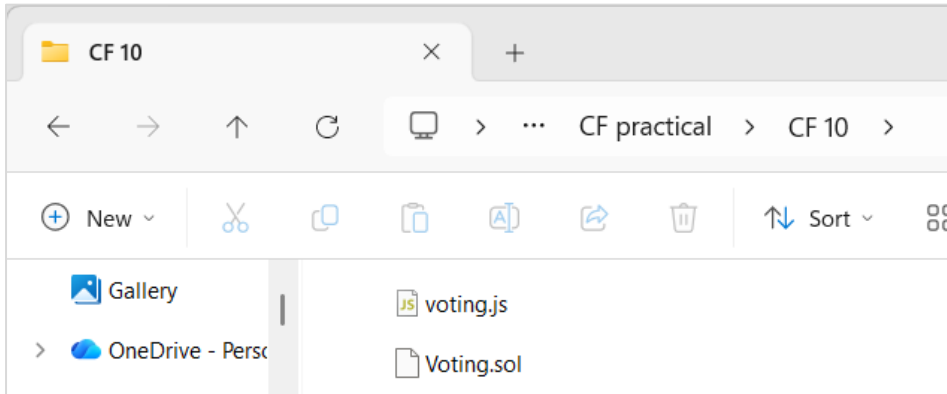


Aim: Aim: Develop a voting application using Ethereum.



voting.js

```
const { ethers } = require("ethers");
const fs = require("fs");
const path = require("path");
const solc = require("solc");
const readline = require("readline");

// =====
// ⚠️ CRITICAL CREDENTIAL CONFIGURATION: Paste your parameters directly here
// =====
const SEPOLIA_RPC_URL = "https://ethereum-sepolia-rpc.publicnode.com";
const PRIVATE_KEY = "6d71ad5277a82774e258f52bf81f0026e378aaead28a3cb8f66e29ce8211d43e";
// =====

// Paste your successful deployed contract address inside these quotes!
let contractAddress = "";

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

const askQuestion = (query) => new Promise((resolve) => rl.question(query, resolve));

async function main() {
  if (PRIVATE_KEY.includes("YOUR_")) {
    console.error("\n ❌ INITIALIZATION ERROR: Update your true private key variable string at the top lines.");
    process.exit(1);
  }

  const provider = new ethers.JsonRpcProvider(SEPOLIA_RPC_URL, {
    name: "sepolia",
    chainId: 11155111
  }, { staticNetwork: true });

  const wallet = new ethers.Wallet(PRIVATE_KEY, provider);
  console.log("\n 🗝️ Connected Cryptographic Signer Wallet Address: ${wallet.address}");
```

```
// DYNAMIC LOCAL COMPILATION ENGINE
console.log("⌚ Compiling 'Voting.sol' locally to map out interface rules...");
const contractPath = path.resolve(__dirname, "Voting.sol");
if (!fs.existsSync(contractPath)) {
  console.error("❌ ERROR: 'Voting.sol' file not found in this folder directory. Keep the file intact here.");
  process.exit(1);
}

const source = fs.readFileSync(contractPath, "utf8");
const input = {
  language: "Solidity",
  sources: { "Voting.sol": { content: source } },
  settings: { outputSelection: { "**": { "**": ["abi", "evm.bytecode.object"] } } }
};

const output = JSON.parse(solc.compile(JSON.stringify(input)));
if (output.errors) {
  output.errors.forEach(err => {
    if (err.severity === "error") {
      console.error("❌ Solidity Compilation Failure:", err.message);
      process.exit(1);
    }
  });
}

const contractData = output.contracts["Voting.sol"]["Voting"];
const contractABI = contractData.abi;
const bytecode = contractData.evm.bytecode.object;

// AUTOMATED SYSTEM DEPLOYMENT FALLBACK
if (!contractAddress || contractAddress.includes("YOUR_")) {
  console.log("\n🌐 No contract address detected. Broadcasting new deployment transaction...");
  const factory = new ethers.ContractFactory(contractABI, bytecode, wallet);
  const contract = await factory.deploy(["Alice", "Bob", "Charlie"]);

  console.log("⌚ Awaiting network mining confirmation block confirmation layers... Please wait.");
  await contract.waitForDeployment();

  contractAddress = await contract.getAddress();
  console.log("\n=====");
  console.log(`🚀 AUTOMATED COMPILATION & DEPLOYMENT COMPLETE!`);
  console.log(`📌 Deployed Contract Address: ${contractAddress}`);
  console.log("=====");
}

// ABI definition is now 100% available and verified outside the deployment block context
const votingContract = new ethers.Contract(contractAddress, contractABI, wallet);
console.log(`🔗 Successfully connected to contract address: ${contractAddress}`);
```

```
// Core CLI Interactive Program Event Loop Dashboard
while (true) {
  console.log("\n=====");
  console.log("📊 SYSTEM BALLOTS CORE BOARD TELEMETRY STATUS");
  console.log("=====");

  try {
    const total = await votingContract.candidatesCount();
    if (Number(total) === 0) {
      console.log(" No metrics recorded on the active blockchain ledger instance.");
    } else {
      for (let i = 1; i <= Number(total); i++) {
        const cand = await votingContract.candidates(i);
        console.log(` 💎 [ID: ${cand.id.toString()}] ${cand.name.padEnd(10)} — Recorded Votes:
${cand.voteCount.toString()} `);
      }
    }
  } catch (err) {
    console.error(" ❌ Node fetching parsing transaction block crash.", err.message);
  }

  console.log("=====");
  console.log(" 1️⃣ Submit a Cryptographic Vote");
  console.log(" 2️⃣ Flush & Hard-Refresh Local Data metrics");
  console.log(" 3️⃣ Terminate Application Console Runtime");
  console.log("=====");

  const choice = await askQuestion("\n👉 Key in an operational execution line number (1-3): ");

  if (choice === "1") {
    const idInput = await askQuestion("👉 Enter the numeric Candidate ID you wish to vote for: ");
    const targetId = parseInt(idInput);

    if (isNaN(targetId) || targetId <= 0) {
      console.log(" ⚠️ Input exception. Key numeric variables listed on scoreboard.");
      continue;
    }

    try {
      console.log(" ⌚ Bundling parameters and applying digital key signatures...");
      const txn = await votingContract.vote(targetId);
      console.log(" 🌀 Broadcast successful! Gas Hash index: ${txn.hash}");
      console.log(" ⌚ Awaiting validation check confirmation passes across nodes... Please wait.");

      await txn.wait();
      console.log(" 🎉 Success! Your vote has been hard-coded securely onto the blockchain block
matrix.");
    }
  }
}
```

```

    } catch (err) {
      if (err.reason) {
        console.error(` ✖ Transaction reverted by contract EVM criteria: ${err.reason}`);
      } else if (err.message && err.message.includes("insufficient funds")) {
        console.error(" ✖ Network block rejection: Your wallet lacks testnet ETH to offset transaction
gas computations.");
      } else {
        console.error(` ✖ Process failure loop crash: ${err.message}`);
      }
    }
  } else if (choice === "2") {
    console.log(" ⌚ Clearing caching arrays and fetching raw variables states...");
    continue;
  } else if (choice === "3") {
    console.log(" 🛑 Uncoupling background terminal event listeners. Terminating instance.");
    rl.close();
    break;
  } else {
    console.log(" ⚠ Input out of parameters. Select numerical parameters between 1 and 3.");
  }
}
}

main().catch((error) => {
  console.error(" 💥 SYSTEM RUNTIME FAULT EXCEPTION CRASH:", error);
  process.exit(1);
});

```

Voting.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

```

```

contract Voting {
  struct Candidate {
    uint256 id;
    string name;
    uint256 voteCount;
  }

  mapping(uint256 => Candidate) public candidates;
  mapping(address => bool) public voters;
  uint256 public candidatesCount;

  event Voted(uint256 indexed candidateId, address indexed voter);

  // Initialize contract with an array of candidate names
  constructor(string[] memory _candidateNames) {
    for (uint256 i = 0; i < _candidateNames.length; i++) {
      addCandidate(_candidateNames[i]);
    }
  }
}

```

```
function addCandidate(string memory _name) private {
    candidatesCount++;
    candidates[candidatesCount] = Candidate(candidatesCount, _name, 0);
}

// Cast a vote for a candidate ID
function vote(uint256 _candidateId) public {
    // 1. Ensure the user hasn't voted already
    require(!voters[msg.sender], "You have already voted.");

    // 2. Ensure the candidate ID is valid
    require(_candidateId > 0 && _candidateId <= candidatesCount, "Invalid candidate ID.");

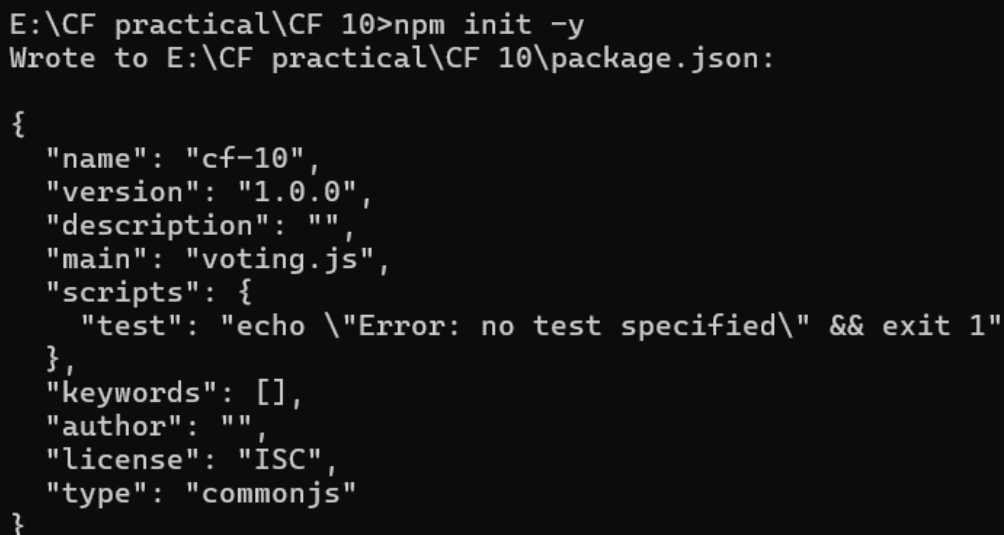
    // 3. Record that the user has voted
    voters[msg.sender] = true;

    // 4. Update candidate vote count
    candidates[_candidateId].voteCount++;

    // Trigger the voting event
    emit Voted(_candidateId, msg.sender);
}
}
```

Step 2: Open PowerShell in that folder.

Run: npm init -y



```
E:\CF practical\CF 10>npm init -y
Wrote to E:\CF practical\CF 10\package.json:

{
  "name": "cf-10",
  "version": "1.0.0",
  "description": "",
  "main": "voting.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Step 2 — Install the required packages

Your voting.js uses: ethers & solc

Run: npm install ethers solc

Check: npm list ethers solc

```

E:\CF practical\CF 10>npm install ethers solc

added 18 packages, and audited 19 packages in 15s

4 packages are looking for funding
  run `npm fund` for details

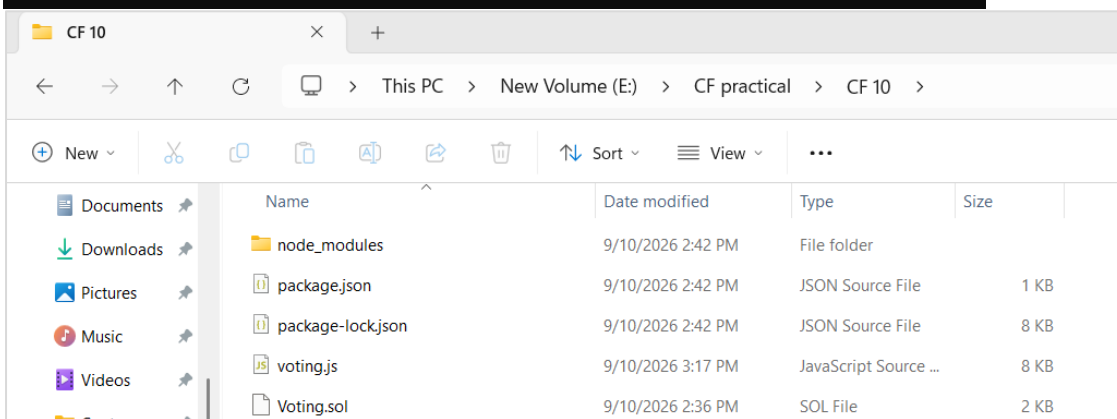
2 vulnerabilities (1 low, 1 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.

E:\CF practical\CF 10>npm list ethers solc
cf-10@1.0.0 E:\CF practical\CF 10
+-- ethers@6.17.0
`-- solc@0.8.36

```



Name	Date modified	Type	Size
node_modules	9/10/2026 2:42 PM	File folder	
package.json	9/10/2026 2:42 PM	JSON Source File	1 KB
package-lock.json	9/10/2026 2:42 PM	JSON Source File	8 KB
voting.js	9/10/2026 3:17 PM	JavaScript Source ...	8 KB
Voting.sol	9/10/2026 2:36 PM	SOL File	2 KB

Step 3 — Check Voting.sol compilation

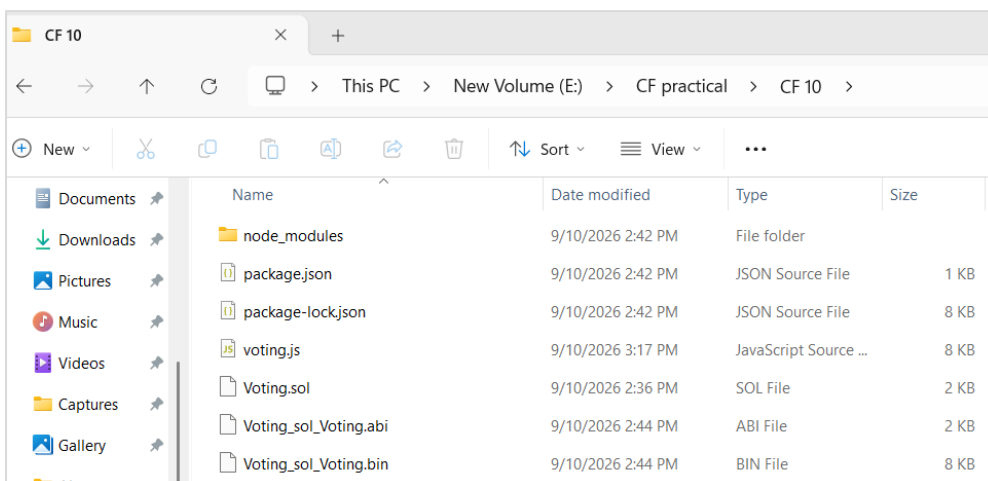
`npx solcjs --bin --abi Voting.sol`

```
E:\CF practical\CF 10>npx solcjs --bin --abi Voting.sol
```

It should also create two new files:

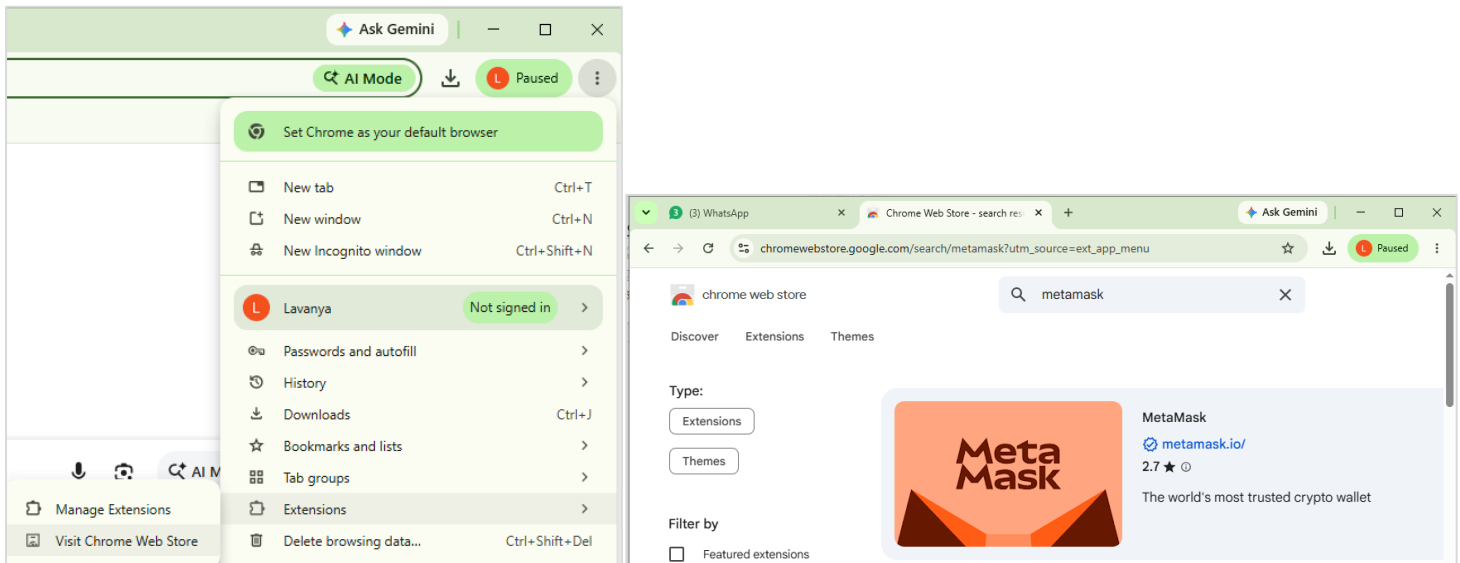
Voting_sol_Voting.abi

Voting_sol_Voting.bin

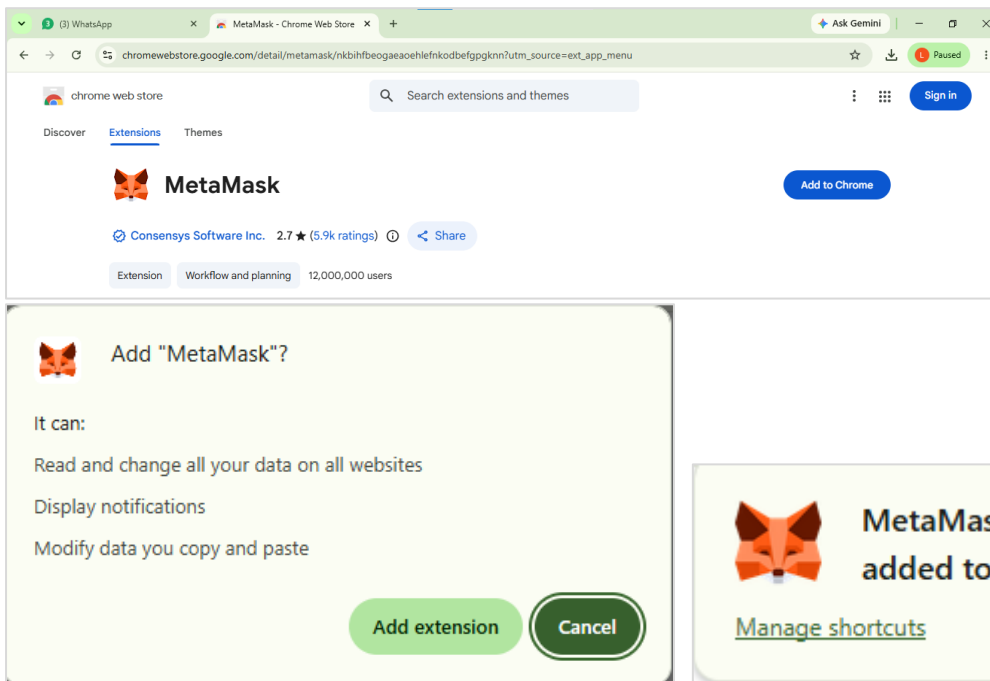


Name	Date modified	Type	Size
node_modules	9/10/2026 2:42 PM	File folder	
package.json	9/10/2026 2:42 PM	JSON Source File	1 KB
package-lock.json	9/10/2026 2:42 PM	JSON Source File	8 KB
voting.js	9/10/2026 3:17 PM	JavaScript Source ...	8 KB
Voting.sol	9/10/2026 2:36 PM	SOL File	2 KB
Voting_sol_Voting.abi	9/10/2026 2:44 PM	ABI File	2 KB
Voting_sol_Voting.bin	9/10/2026 2:44 PM	BIN File	8 KB

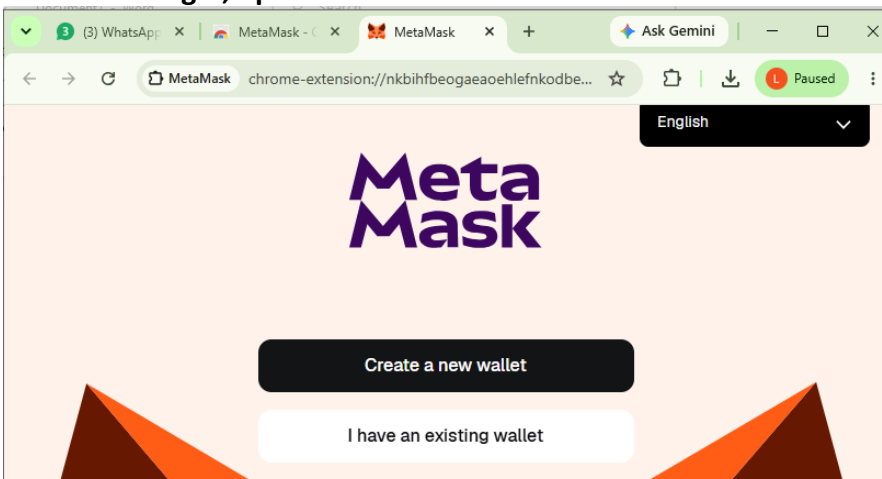
Open Google Chrome and click on 3 dots -> Extensions
search for **MetaMask official website**.



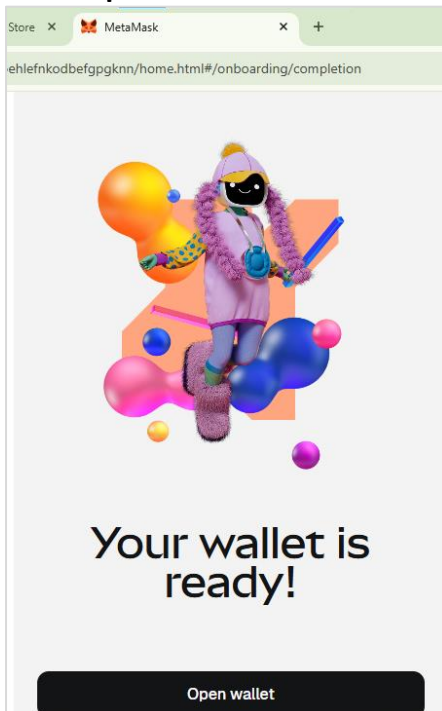
Install the MetaMask extension in Chrome. Click on Add to chrome



After installing it, open MetaMask. Click on Create a new wallet



Click **“Open wallet”**.



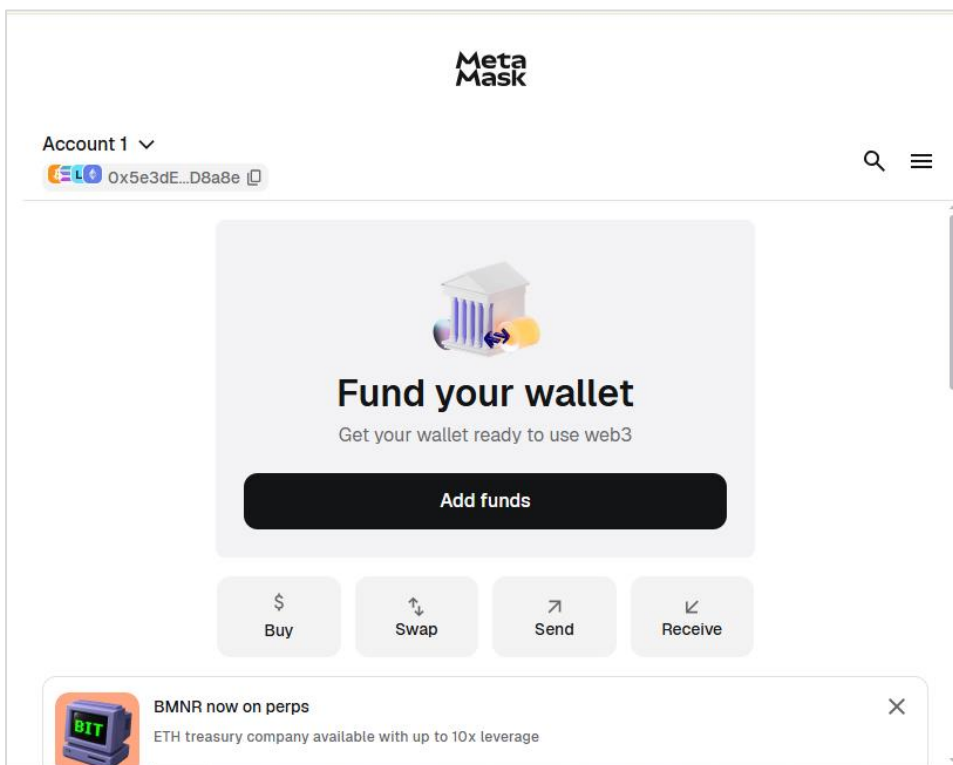
Step : Copy your wallet address

In MetaMask, look near the top of the wallet. You should see an address starting with: 0x...

Click the **copy icon** next to the address.

This is your **public wallet address**.

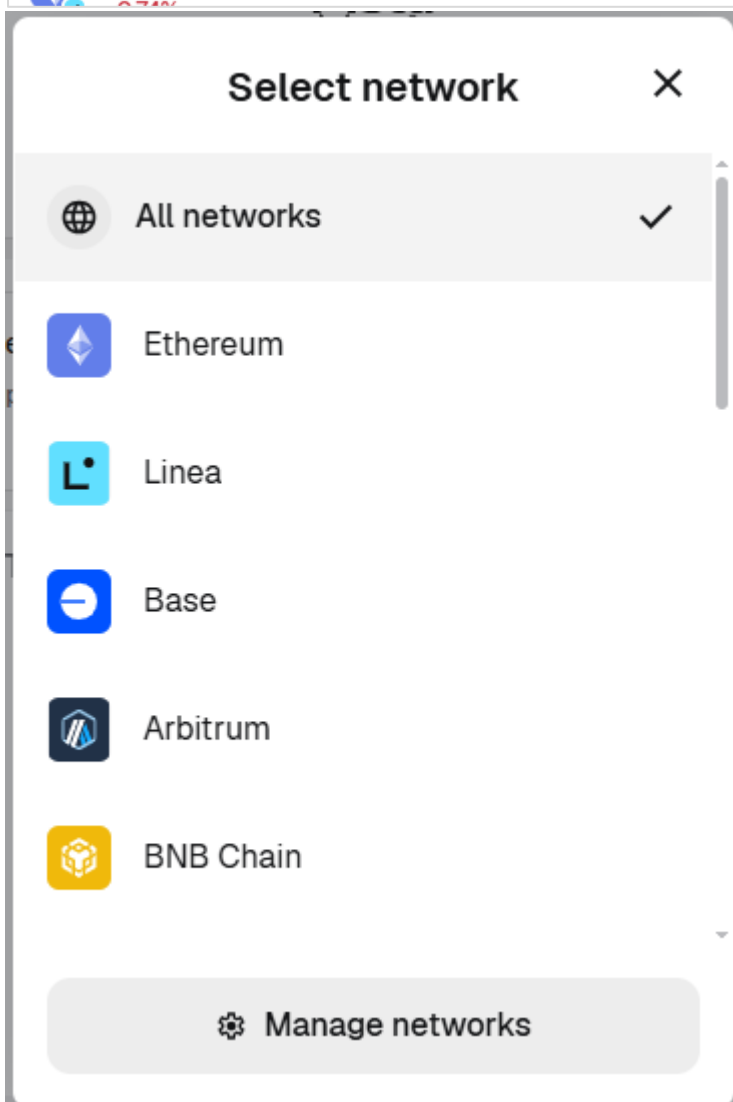
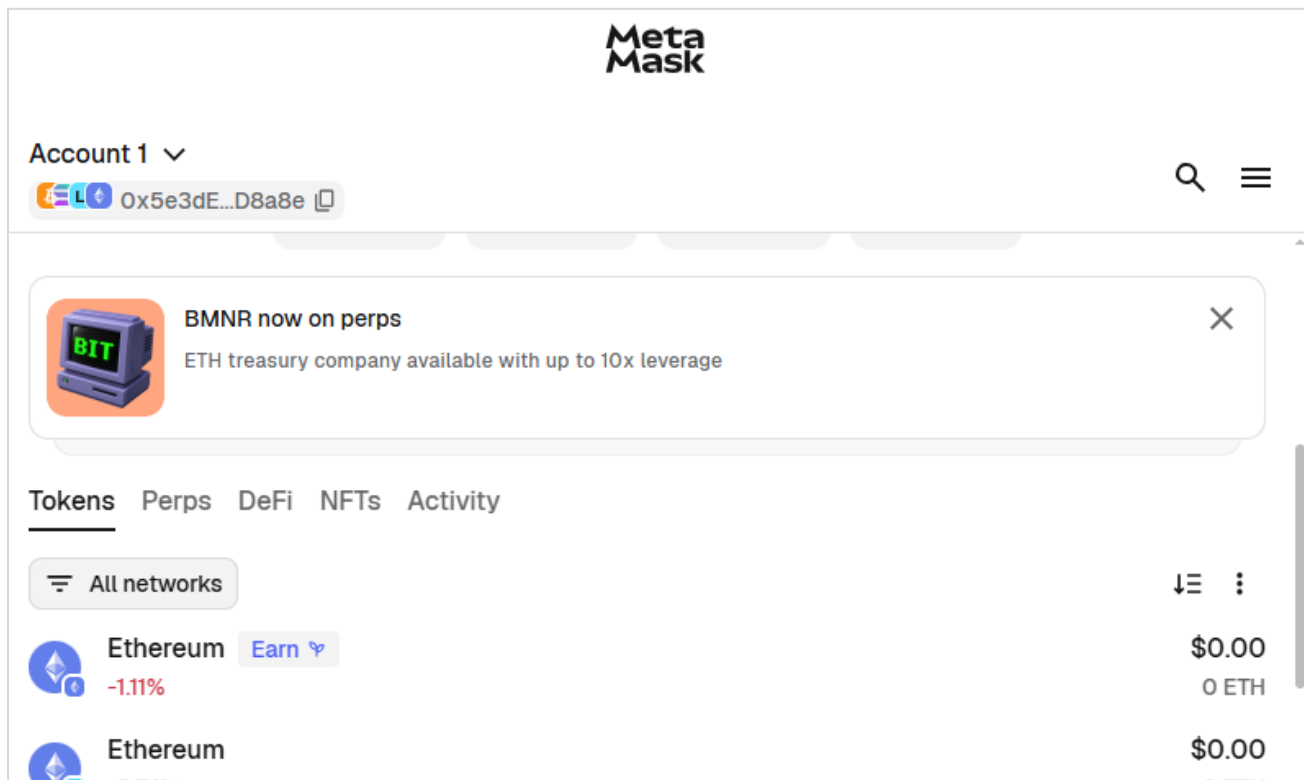
Public address: 0x4208C7EfCe7137B4F8871d17cF7d5813d7eDdb92



Step: Turn on test networks

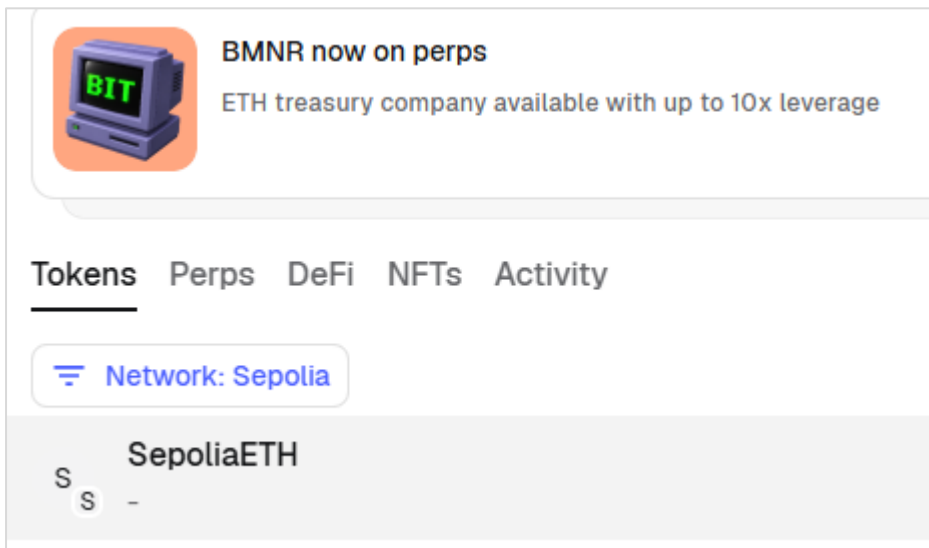
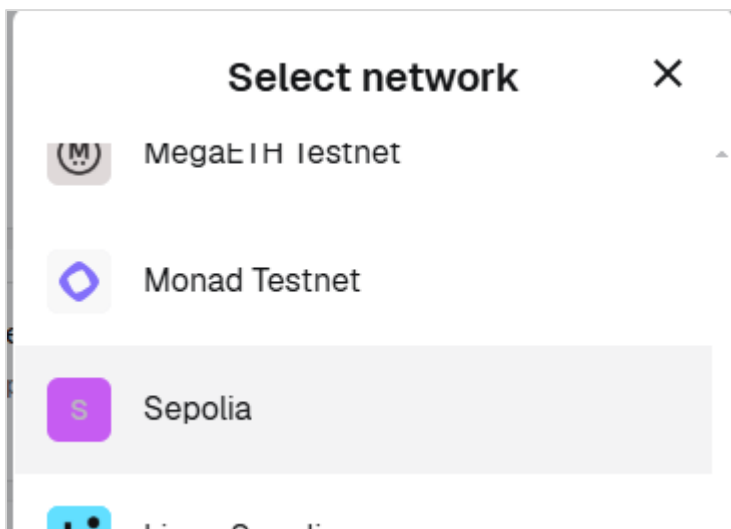
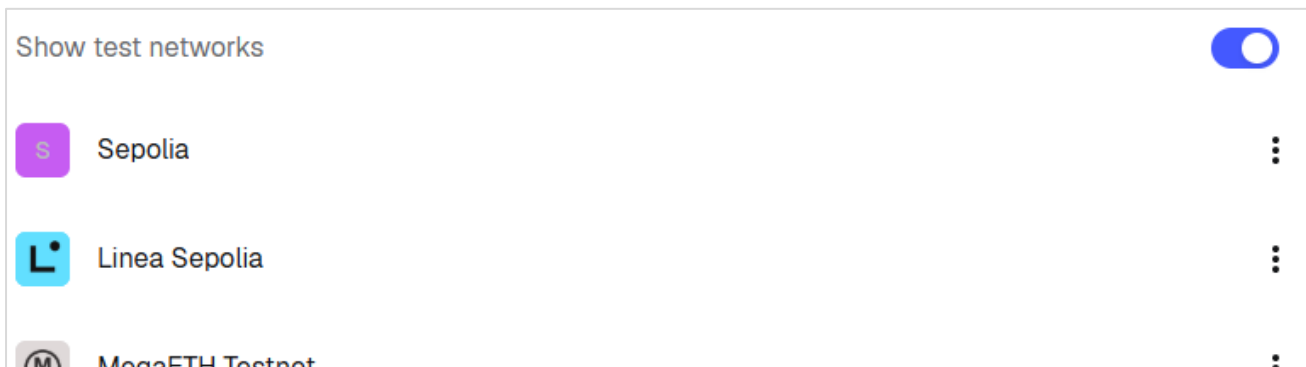
In MetaMask:

Click the **All networks**.



Look for **“Show test networks”** & Turn it **ON**.

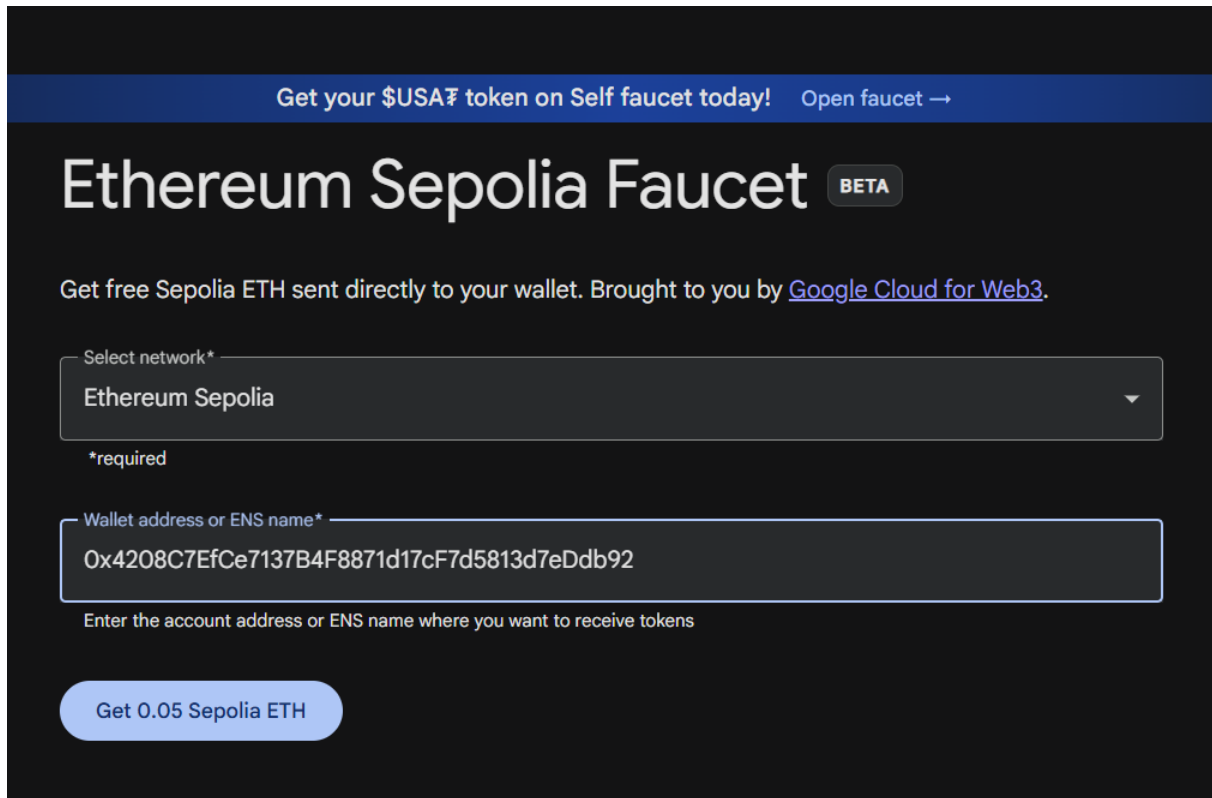
Open the network list again. You should now see **Sepolia**. Click **Sepolia**.



Step : Get Sepolia test ETH

Paste the **wallet address you copied from MetaMask**. Click the button to request/send test ETH.

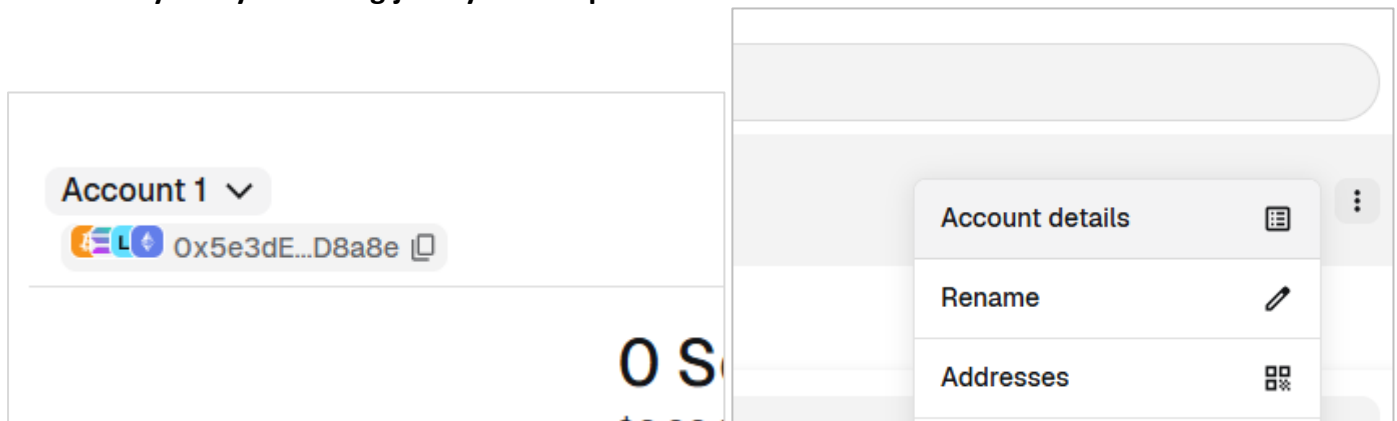
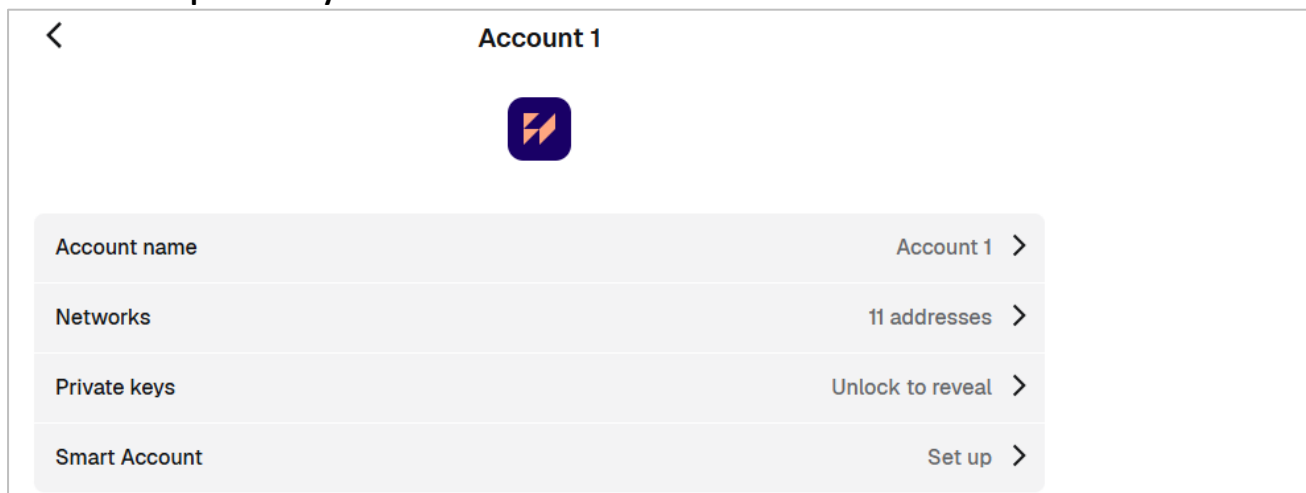
Public address: 0x4208C7EfCe7137B4F8871d17cF7d5813d7eDdb92

**Step: Put your new private key into voting.js**

Click the three dots / account menu. Go to Account details.

Confirm with your MetaMask password. Copy the private key.

Paste it only into your voting.js on your computer.

**Choose Show private key**

Confirm with your MetaMask password.

 **Account 1 / Private keys**

 **Don't share your private key**
This key grants full control of your account for the associated chain. [Learn more](#)

Enter your password

Cancel

Confirm

Copy the private key. (Ethereum)

 **Account 1 / Private keys**

 **Don't share your private key**
This key grants full control of your account for the associated chain. [Learn more](#)

 **Ethereum**
0x5e3de...d8a8e

 **Linea**
0x5e3de...d8a8e

 **Base**
0x5e3de...d8a8e



Paste it only into code of voting.js & Save voting.js

```
// 🚨 CRITICAL CREDENTIAL CONFIGURATION: Paste your parameters directly here
// =====
const SEPOLIA_RPC_URL = "https://ethereum-sepolia-rpc.publicnode.com";
const PRIVATE_KEY = "6d71ad5277a82774e258f52bf81f0026e378aaead28a3cb8f66e29ce8211d43e";
// =====
```

Run the voting application

node voting.js

```
E:\CF practical\CF 10>node voting.js

🔑 Connected Cryptographic Signer Wallet Address: 0x4208C7EfCe7137B4F8871d17cF7d5813d7eDdb92
⌚ Compiling 'Voting.sol' locally to map out interface rules...

🌐 No contract address detected. Broadcasting new deployment transaction...
⌚ Awaiting network mining confirmation block confirmation layers... Please wait.

=====
🎉 AUTOMATED COMPILATION & DEPLOYMENT COMPLETE!
📌 Deployed Contract Address: 0xE36A63493b853F55645Fe9F043e5D07bB49a319f
=====
🔗 Successfully connected to contract address: 0xE36A63493b853F55645Fe9F043e5D07bB49a319f

=====
📊 SYSTEM BALLOTScore BOARD TELEMETRY STATUS
=====
♦ [ID: 1] Alice      - Recorded Votes: 0
♦ [ID: 2] Bob       - Recorded Votes: 0
♦ [ID: 3] Charlie   - Recorded Votes: 0
=====
❑ 1 Submit a Cryptographic Vote
❑ 2 Flush & Hard-Refresh Local Data metrics
❑ 3 Terminate Application Console Runtime
=====

📌 Key in an operational execution line number (1-3):
```

Candidate Selection for Voting Transaction**Input:** Candidate ID 1 (Alice) was selected for voting.**Output:** Vote was successfully recorded on the Ethereum blockchain and Alice's vote count became 1.

```
📌 Key in an operational execution line number (1-3): 1
👉 Enter the numeric Candidate ID you wish to vote for: 1
⌚ Bundling parameters and applying digital key signatures...
🔗 Broadcast successful! Gas Hash index: 0x18523bda57bb4473a35cc4f8253d5de17cb6a8e343170da88dc89491cb90ac55
⌚ Awaiting validation check confirmation passes across nodes... Please wait.
🎉 Success! Your vote has been hard-coded securely onto the blockchain block matrix.

=====
📊 SYSTEM BALLOTScore BOARD TELEMETRY STATUS
=====
♦ [ID: 1] Alice      - Recorded Votes: 1
♦ [ID: 2] Bob       - Recorded Votes: 0
♦ [ID: 3] Charlie   - Recorded Votes: 0
=====
❑ 1 Submit a Cryptographic Vote
❑ 2 Flush & Hard-Refresh Local Data metrics
❑ 3 Terminate Application Console Runtime
=====
```

Prevention of Multiple Voting**Input:** Select option 1 (Submit Vote) and enter Candidate ID 2 (Bob).**Output: Transaction reverted: You have already voted.**This demonstrates that the same wallet **cannot vote twice**.

```
📌 Key in an operational execution line number (1-3): 1
👉 Enter the numeric Candidate ID you wish to vote for: 2
⌚ Bundling parameters and applying digital key signatures...
❌ Transaction reverted by contract EVM criteria: You have already voted.
```